

System Analizy Rynku Kryptowalut w Czasie Rzeczywistym

Podjęcie Champion/Challenger

Jakub Wojna

Wydział Nauk Ekonomicznych
Uniwersytet Warszawski

15 grudnia 2025

Pytanie Badawcze

Czy dynamiczny system wielu modeli (*Ensemble*) radzi sobie lepiej z dużą zmiennością rynku kryptowalut niż pojedynczy, statyczny algorytm?

Główne cele projektu:

- Budowa palety danych odpornego na awarie w środowisku Databricks.
- Rozwiązanie problemu niestacjonarności danych.
- Implementacja 5 modeli matematycznych rywalizujących na żywo.

Architektura Systemu (Micro-Batching)

Wyzwanie: Blokada zapisu plików w Databricks Serverless uniemożliwiła klasyczny streaming.

Rozwiązanie: Architektura Delta Lake

API Coinbase $\xrightarrow{5s}$ **Producent** $\xrightarrow{\text{APPEND}}$ Tabela Delta $\xrightarrow{\text{READ}}$ **Konsument**

- **Storage:** Zarządzane Tabele Delta (ACID).
- **Ingestia:** Zadanie w tle (Job) pobierające cenę i wolumen.
- **Inferencja:** Pętla analityczna w Pythonie (Micro-batches).

Modele trenowane na cenie surowej są podatne na *Concept Drift*. Zastosowano transformację do **Log-Zwrotów**.

Wzór na Logarytmiczny Zwrot (r_t)

$$r_t = \ln\left(\frac{P_t}{P_{t-1}}\right) = \ln(P_t) - \ln(P_{t-1})$$

Zalety:

- Addytywność w czasie.
- Stacjonarność (średnia dąży do 0, wariancja jest stabilniejsza).
- Niezależność od poziomu ceny (model działa tak samo przy 2800\$ jak i 3500\$).

Problem: Modele oparte na odległości (jak k-NN) wymagają ujednolicenia skali zmiennych. Bez tego Wolumen (liczby w milionach) zagłuszyłby Cenę (liczby w tysiącach).

Standaryzacja (Z-Score)

Dla każdej cechy x (np. `returns_sma`, `volume`):

$$z = \frac{x - \mu}{\sigma}$$

Gdzie:

- μ – średnia z próby treningowej.
- σ – odchylenie standardowe z próby treningowej.

1. Regresja Liniowa (Baseline)

$$y = X\beta + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \sigma^2)$$

2. Bayesian Ridge Regression Wprowadza priorytetowy rozkład prawdopodobieństwa dla wag w , co zapobiega przeuczeniu.

Funkcja celu (maksymalizacja a posteriori):

$$\min_w (||y - Xw||_2^2 + \lambda ||w||_2^2)$$

Gdzie λ jest hiperparametrem regularyzacji (precyzją rozkładu a priori).

XGBoost: Extreme Gradient Boosting

Model zespołowy oparty na drzewach decyzyjnych. Minimalizuje funkcję celu z regularyzacją.

Funkcja Celu (Objective Function)

$$\mathcal{L}(\phi) = \sum_i l(\hat{y}_i, y_i) + \sum_k \Omega(f_k)$$

Człon regularyzacyjny $\Omega(f)$ (kluczowy w projekcie):

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \|w\|^2 + \alpha \|w\|_1$$

- l : Funkcja straty (u mnie w projekcie MAE/MSE).
- T : Liczba liści w drzewie.
- λ, α : Parametry regularyzacji L2 i L1 (użyte w Hyperopt).

k-Najbliższych Sasiadów (k-NN)

Model nieparametryczny. Predykcja to ważona średnia k historycznych momentów najbardziej podobnych do obecnego.

Metryka Odległości (Euklidesowa)

$$d(x, x_{hist}) = \sqrt{\sum_{j=1}^m (x_j - x_{hist,j})^2}$$

Estymator regresji:

$$\hat{y} = \frac{\sum_{i=1}^k w_i y_i}{\sum_{i=1}^k w_i}, \quad \text{gdzie } w_i = \frac{1}{d(x, x_i)}$$

Dzięki temu model szybko reaguje na powtarzalne wzorce rynkowe.

Model hybrydowy łączący ML z inżynierią finansową.

Stochastyczne Równanie Różniczkowe (SDE)

$$dS_t = \mu S_t dt + \sigma S_t dW_t + S_t dJ_t$$

Składniki modelu w naszej implementacji:

- μ (**Drift**): Pobierany z predykcji modelu XGBoost.
- dW_t (**Dyfuzja**): Proces Wienera (szum gaussowski $\mathcal{N}(0, \sigma^2)$).
- dJ_t (**Skok**): Proces Poissona symulujący nagłe szoki cenowe.

Wynik końcowy to średnia z 1000 symulowanych ścieżek.

w teoria dobór modelu a Wielkość Próby powinny wyglądać następująco

Małe zbiory danych ($N < 100$)

Zwycięzca: Bayesian Ridge / Linear

- **Dlaczego?** Modele o wysokim obciążeniu (*High Bias*), ale niskiej wariancji.
- **Zaleta:** Podejście Bayesowskie wprowadza rozkłady a priori które stabilizują model, gdy brakuje trendu w danych.
- **Ryzyko:** XGBoost szybko uległby przeuczeniu.

Duże zbiory danych ($N > 1000$)

Zwycięzca: XGBoost / k-NN

- **Dlaczego?** Modele o niskim obciążeniu, zdolne do modelowania nieliniowości.
- **Zaleta:** Wraz ze wzrostem N , błąd wariancji maleje, pozwalając modelom wyłapywać subtelniejsze wzorce rynkowe.
- **Ryzyko:** k-NN staje się kosztowny obliczeniowo.

Teoria: Funkcje Straty (Część 1 - RMSE)

Wybór metryki optymalizacji determinuje zachowanie modelu wobec "szpilek" cenowych (Outliers).

RMSE (Root Mean Squared Error)

Preferowane przez modele Liniowe i Bayesowskie.

$$RMSE = \sqrt{\frac{1}{n} \sum (y - \hat{y})^2}$$

- **Charakterystyka:** Podnosi błędy do kwadratu. Bardzo mocno kara duże pomyłki.
- **Zastosowanie:** Gdy chcemy unikać outlierów, ale akceptujemy mały szum w tle.

Teoria: Funkcje Straty (Część 2 - MAE)

Alternatywne podejście stosowane w modelach odpornych na szum (Robust).

MAE (Mean Absolute Error)

Zastosowane w modelu XGBoost (Robust).

$$MAE = \frac{1}{n} \sum |y - \hat{y}|$$

- **Charakterystyka:** Traktuje błędy liniowo. Jest bardziej odporna na wartości odstające.
- **Zastosowanie:** Idealne w krypto, gdzie występują cena biega tak jak chce. Model generalnie ignoruje chwilowe outliersy i podąża za głównym trendem (w tym wypadku medianą).

Teoria: Kompromis Obciążenie-Wariancja

Dlaczego używamy systemu Champion/Challenger? Ponieważ żaden model nie jest idealny w każdym scenariuszu.

Model	Charakterystyka	Kiedy wygrywa?
Regresja Liniowa	High Bias, Low Variance	Rynek stabilny (Trend boczny). Model ignoruje
k-NN	Low Bias, High Variance	Powtarzalne wzorce lokalne. Szybka reakcja na
XGBoost	Balanced (Regularized)	Rynek zmienny (Volatile). Wykrywa nieliniowość

Wniosek: Błąd całego systemu jest niższy niż błąd pojedynczego składnika, ponieważ błędy modeli nieskorelowanych znoszą się wzajemnie.

Dynamika Uczenia: Dominacja XGBoost nad Modelem Stochastycznym

Kluczowa obserwacja: Relacja skuteczności między modelami zmienia się drastycznie wraz z **wielkością próbki danych (N)**.

Mała Próbką (Cold Start)

Remis / Przewaga Stochastyczna

- Gdy danych jest mało, XGBoost ma za mało przykładów, by wykryć nieliniowości (niedouczenie).
- Model Stochastyczny radzi sobie "nieźle", ponieważ po prostu losuje ścieżki zgodnie ze zmiennością wariancji, co jest bezpiecznym przybliżeniem (w kontekście przyjętych miar).

Duża Próbką ($N \uparrow$)

Dominacja XGBoost

- Przy dużej liczbie danych XGBoost zaczyna rozpoznawać ukryte wzorce, których nie widzi symulacja Monte Carlo.
- Błąd modelu ML maleje (uczenie się), podczas gdy błąd modelu Stochastycznego pozostaje stały (ograniczony przez losowość rynku).

Wniosek: W długim okresie *Pattern Recognition* (ML) wygrywa z hipotezą błędzenia losowego

Analiza Wyników na Żywo (Snapshot)

Analiza zachowania systemu dla ceny ETH = 3012.41 USD.

Zwycięzca: Regresja Liniowa

- **RMSE: 0.4125** (Najniższy błąd)

Interpretacja dla próbki $N = 100$: Dla tak **małej próbki danych** (okno ok. 8 minut) i stabilnego trendu, najlepiej dostosował się **Model Liniowy**.

Złożone modele (k-NN) przestrzeliły prognozę - RMSE 1.19, interpretując lokalny szum jako istotną zmianę, podczas gdy model liniowy prawidłowo zignorował odchylenia.

Tabela Wyników (z konsoli):

Model	Prognoza	RMSE
Linear	3012.00	0.4125
Bayes	3012.00	0.4125
XGBoost	3011.99	0.4169
Stoch	3012.03	0.4168
k-NN	3011.22	1.1910

Analiza Wyników na żywo

Analiza zachowania systemu przy starcie lub restarcie (brak historii).

Zwycięzca: Model Stochastyczny

- **RMSE: 1.4523** (Najniższy błąd)

Interpretacja dla próbki $N < 50$: W warunkach **skrajnego braku danych**, modele uczące się (XGBoost, k-NN) próbują znaleźć wzorce w szumie, co prowadzi do dużych błędów ($\text{RMSE} > 2.0$).

Model Stochastyczny nie "uczy się" na siłę, lecz generuje rozkład prawdopodobieństwa oparty na bieżącej zmienności (σ). W chaosie losowość okazuje się najlepszym predyktorem.

Tabela Wyników:

Model	Prognoza	RMSE
Linear	3049.87	2.1542
XGBoost	3064.92	2.8471
Stoch	3042.15	1.4523
k-NN	3079.12	3.0988
Bayes	3048.64	2.0519

Analiza Wyników na żywo

Analiza po przetworzeniu ponad 10000 próbek (kilka dni ciągłej pracy).

Zwycięzca: XGBoost (Champion)

• RMSE: 0.4009

Interpretacja dla próbki $N > 10000$:

Zadziałało "**Prawo wielkich liczb**". Przy tak ogromnym zbiorze, XGBoost nauczył się subtelnych, nieliniowych mikrowzorców, niewidocznych dla człowieka.

Podczas gdy błąd Modelu Stochastycznego ustabilizował się na poziomie losowości rynku (0.65), model ML sukcesywnie minimalizował błąd, "odszumiając" sygnał.

Tabela Wyników:

Model	Prognoza	RMSE
Linear	3010.48	0.6124
XGBoost	3010.12	0.4009
Stoch	3010.73	0.6511
k-NN	3010.39	0.5894
Bayes	3010.51	0.6097

Analiza Wyników, Flash Crash

Sytuacja nagłego załamania rynku (spadek o 3% w 5 minut).

Zwycięzca: k-NN (Historyk)

• RMSE: 0.8491

Interpretacja (Powtarzalność Historii): W momencie paniki rynkowej modele globalne (Linear, XGBoost) reagują z opóźnieniem lub uśredniają trend.

k-NN zadziałał najlepiej, ponieważ odnalazł w historii podobny krach (sąsiada) i "przypomniał sobie", jak rynek zachował się wtedy (np. szybkie odbicie - *Dead Cat Bounce*).

Wykorzystał pamięć historyczną tam, gdzie statystyka zawiodła.

Tabela Wyników:

Model	Prognoza	RMSE
Linear	2950.12	3.1982
XGBoost	2910.45	1.5034
Stoch	2904.88	1.7955
k-NN	2885.23	0.8491
Bayes	2944.91	3.0976

Obserwacje z działania systemu:

- 1 **XGBoost** najczęściej wygrywa w okresach trendowych dzięki nieliniowości i dużej ilości danych.
- 2 **Modele Liniowe** są stabilniejsze w okresach bocznych (niska wariancja).
- 3 **Log>Returns** skutecznie wyeliminowały problem *Concept Drift*.

Raport AI

W projekcie wykorzystano Generatywną AI do:

- Optymalizacji kodu PySpark (obejście blokad Serverless).
- Optymalizacji implementacji XGBoost.
- Poprawianie opisów modeli pod kątem dokładności przekazu danych.
- Pomoc w syntaxie kodu LaTeX, poprawianie błędów językowych/stylistycznych.

Dziękuję za uwagę!

Pytania?